¿En qué consiste este juego?

El diagrama de la imagen no representa un juego en particular (como el Ta-Te-Ti o el ajedrez), sino la **representación abstracta de un juego de dos jugadores, de suma cero, con información perfecta y turnos alternados**.

En este tipo de juegos:

- **MAX (Jugador 1 / Líneas Azules):** Querés tomar las decisiones que te lleven al **resultado más alto posible** (ganar o maximizar tus puntos).
- **MIN (Jugador 2 / Líneas Rojas):** El rival quiere obligarte a obtener el **resultado más bajo posible** (para ganar él o minimizar tus puntos).
- **Las hojas (números del fondo):** Representan el puntaje de utilidad o la "ganancia" de la posición al final de una secuencia de jugadas.

¿Por qué se evalúa de abajo hacia arriba?

Se resuelve de abajo hacia arriba (técnicamente llamado **evaluación en post-orden** o **backtracking**) por una razón lógica fundamental:

Para saber qué te conviene elegir HOY, primero necesitas saber qué consecuencias tendrán tus decisiones en el FUTURO.

1. **Predicción del comportamiento futuro:** No podes saber cuánto vale la rama de la izquierda en la Raíz si no sabes qué responderá tu rival ante esa jugada.
2. **Propagación del valor real:** Cada jugador elige de manera óptima en su turno. Por lo tanto, el valor de una decisión en un nodo superior **depende directamente de la respuesta óptima del jugador en el nivel inferior**.

¿Es porque antes ya se construyó el árbol de decisión?

Sí, exactamente.

Para aplicar el algoritmo Min-Max (y la poda Alfa-Beta) se asume un proceso en **dos fases relativas**:

1. **Fase de Exploración (Arriba hacia Abajo):** La computadora genera las posibles jugadas recursivamente desde la posición actual (raíz) bajando por todas las ramificaciones hasta llegar a los estados finales o a una profundidad límite (las hojas).
2. **Fase de Evaluación e Inferencia (Abajo hacia Arriba):** Una vez alcanzadas las hojas, se les asigna un valor numérico. Luego, mediante **backtracking**, esos valores comienzan a "subir" hacia la raíz:
    - Los nodos **MIN** suben el valor más chico de sus hijos.
    - Los nodos **MAX** suben el valor más grande de sus hijos.

Nota sobre la Poda Alfa-Beta: En la práctica programada, ambas fases ocurren en simultáneo gracias a la recursividad (Depth-First Search o Búsqueda en Profundidad). A medida que el código va explorando hacia abajo, va calculando y **podando ramas enteras antes de terminarlas de construir o evaluar**, lo que ahorra tiempo de cómputo y memoria.

Para entender la **poda Alfa-Beta** sin enredarnos con fórmulas matemáticas, la clave está en mirar el árbol con una sola idea en mente: **no perder el tiempo analizando opciones que sabes que el rival o vos van a descartar seguro**.

Vamos a repasar los momentos exactos de la imagen donde la gente más se confunde y por qué ocurren los cortes.

1. La Regla del "Límite" (Regla de Oro)

Durante todo el juego, la computadora mantiene dos números grabados en memoria:

- **(Alfa):** Es lo **mínimo que ya tenés asegurado vos (MAX)** en el juego. Si encontrás una jugada que te da un valor mayor, actualizás Alfa.
- **(Beta):** Es lo **máximo que te va a dejar sacar el rival (MIN)**. Si él encuentra una jugada que te perjudica más (un número más bajo), actualiza Beta.

¿Cuándo se corta (poda) una rama?

En el momento en que **Beta (es menor o igual) Alpha**

Es decir: cuando descubrís que en la rama actual el rival te puede forzar a un puntaje que es **igual o peor** que el que vos ya te aseguraste en otro lado.

2. Desglose detallado de los 3 casos clave de la imagen

CASO 1: El primer corte (Extrema izquierda, la X abajo a la izquierda)

1. Analizás la primera rama: el nodo **MIN** compara 8 y 4, y como quiere el número más chico, se queda con el **4**.
2. Arriba de él, el nodo **MAX** guarda ese valor: **ya tiene asegurado un 4** (Alpha = 4).
3. Pasamos al siguiente nodo **MIN** de la derecha. Empieza leyendo el primer número disponible: un **2**.
4. **¿Qué piensa la computadora en ese instante?**

- El nodo es un **MIN** (rival). El rival siempre elegirá el menor número posible.
- Como ya encontró un 2, la decisión final de este nodo **MIN** será 2 o un número más chico si apareciera después. Jamás va a ser mayor a 2.
- Arriba, el jugador **MAX** ya tiene asegurado un **4** en la otra rama.
- **Pregunta clave:** ¿El jugador MAX va a elegir alguna vez esta rama que le va a dar a lo sumo un 2, teniendo ya un 4 al lado? **No, nunca.**
- **Consecuencia:** Es inútil leer el número que sigue (el 7). **Se poda inmediatamente.**

CASO 2: El corte del nodo H (En el medio a la izquierda)

1. Siguiendo el recorrido, el nodo **MIN** analiza 8 y 5, y se queda con el **5**.
2. Su padre (**MAX**) recibe ese 5 y lo guarda Alpha = 5.
3. Pero miremos qué pasa en el nivel superior (el nodo **MIN** más grande de la izquierda):
   - A su izquierda, ese nodo **MIN** ya tenía asegurado un **4**.
   - Ahora ve que a su derecha, el nodo **MAX** ya llegó a **5** (o tal vez más).
   - Como este jugador es **MIN**, él busca el número más chico. Entre un 4 que tiene seguro a la izquierda y un 5 (o más) que viene de la derecha, **se va a quedar con el 4**.
   - **Pregunta clave:** Si el jugador MIN ya decidió que elegirá la opción del 4, ¿importa seguir explorando la derecha del nodo MAX (el nodo H)? **No, porque cualquier cosa que sume ahí no cambiará la decisión de MIN.**
   - **Consecuencia: Se poda todo el nodo H** (con sus hojas 2 y 7).

CASO 3: El corte masivo del lado derecho (El nodo K)

Este es el corte más potente del gráfico y el que más tiempo ahorra:

1. La **Raíz (MAX)** de todo el árbol termina de procesar todo el lado izquierdo y determina que **su mejor opción hasta ahora es un 4** Alpha = 4.
2. Empieza a analizar el lado derecho. Baja por el árbol hasta encontrar que la primera subrama de la derecha devuelve un **3**.
3. El nodo **MIN** del nivel superior de la derecha toma ese **3** Beta = 3.
4. **Pensamiento final del algoritmo:**
   - El nodo **MIN** de la derecha te garantiza que desde su lado vas a recibir como mucho un **3** (porque si sigue buscando, solo elegirá números menores o iguales a 3).
   - La **Raíz (MAX)** está arriba del todo y tiene que elegir entre la izquierda o la derecha.
   - A la izquierda, la Raíz ya tiene un **4** asegurado.
   - A la derecha, el rival (**MIN**) ya le bloqueó el camino y como mucho lo va a dejar sacar un **3**.
   - **Conclusión:** La Raíz **jamás va a elegir el lado derecho**.

- **Consecuencia:** Se cancela la exploración de toda la mitad derecha restante. **Se podan el nodo K, el nodo N, el nodo O y las 4 hojas del final** sin tocarlos siquiera.

Resumen para recordar fácil

Si estás en un nodo...	**Y el jugador es...**	**Pensás así:**
MIN (Rival)	Busca el valor mínimo	*"Si me encuentro un número más bajo que el límite que ya tiene MAX arriba, MAX nunca va a venir por acá. **Podo**."*
MAX (IA)	Busca el valor máximo	*"Si me encuentro un número más alto que el límite que ya se aseguró MIN arriba, MIN nunca me va a dejar elegir este camino. **Podo**."*